

1 RideFlow Engineering, Operations, and Self-Study Handbook

Project: RideFlow

Audience: Founder, future developer, and operations owner

Current stack: React, TypeScript, Vite, Tailwind, Express, tRPC, Drizzle ORM, MySQL/TiDB, Manus OAuth, S3-compatible storage, Vitest

Initial admin email: njengastephen112@gmail.com

Repository: <https://github.com/raiven427/ride-flow>

1.1 1. What RideFlow is

RideFlow is a ride-hailing marketplace prototype designed around upfront fares, driver choice, safety tools, transparent fees, and a five-percent platform commission. The application currently contains a polished customer and driver experience, full-stack foundations, profile and driver-document storage, server-side Kenya fare calculation, admin controls, and tests. It is not yet a complete transport operation: live maps, driver dispatch, realtime location, production payments, payouts, formal verification operations, and legal launch controls still need to be completed before real rides or real money are processed.

The most important rule is that the browser is not trusted. A browser can display a fare and submit a request, but the server must recalculate prices, check permissions, create ledger entries, process payments, and decide what data a user may access.

1.2 2. The software you should learn

You do not need to finish a university degree before continuing RideFlow. You need a structured foundation and practice. Study the subjects in the order below.

Subject	What to learn	How it appears in RideFlow
HTML and CSS	Semantic structure, responsive design, forms, accessibility, layout	<code>client/index.html</code> , <code>client/src/index.css</code>
JavaScript	Variables, functions, objects, promises, events, modules	UI interactions and browser state
TypeScript	Types, unions, generics, async functions, compiler errors	All <code>.ts</code> and <code>.tsx</code> files
React	Components, props, state, hooks, effects, controlled forms	<code>client/src/App.tsx</code> , <code>client/src/pages/Home.tsx</code>
HTTP and APIs	Requests, responses, status codes, cookies, JSON, webhooks	tRPC procedures and provider integrations

Subject	What to learn	How it appears in RideFlow
SQL	Tables, primary keys, foreign keys, indexes, transactions, migrations	drizzle/schema.ts and database migrations
Backend engineering	Authentication, authorization, validation, logging, error handling	server/routers.ts, server/db.ts
Security	Least privilege, secrets, hashing, CSRF, rate limits, audit logs	Admin, upload, and payment protection
Testing	Unit, integration, authorization, failure-path tests	server/*.test.ts
Git	Commits, branches, remotes, pull, push, rollback	GitHub repository workflow
Operations	Backups, monitoring, incident response, deployments	Production readiness and support

A practical learning sequence is: build small JavaScript forms; learn TypeScript by typing those forms; build a React page; create a small Express API; store records in SQL; add authentication; then add tests and deployment. Read code actively: change one small feature, run the tests, inspect the diff, and commit it.

1.3 3. Repository map

The repository is organized as follows.

```

client/
  index.html      Browser entry document
  src/
    main.tsx      React and tRPC providers
    App.tsx        Application shell and routes
    pages/Home.tsx RideFlow customer, driver, and admin interface
    components/    Reusable UI components
    contexts/      Theme and application contexts
    hooks/         Reusable React hooks
    lib/trpc.ts    Typed client for server procedures
    index.css      Visual system and responsive styles
  server/
    _core/         Framework plumbing; avoid casual edits
    db.ts          Database helpers and persistence logic
    fare.ts        Pure Kenya fare calculation rules
    routers.ts     tRPC API procedures and permissions

```

storage.ts	S3-compatible storage helper
*.test.ts	Vitest coverage
shared/	Shared types and constants
drizzle/	
schema.ts	Database schema source of truth
migrations/	Generated migration history
docs/	Operations guides and PDFs
package.json	Commands and dependencies
vite.config.ts	Development/build configuration
vitest.config.ts	Test configuration

The central request path is: React component -> typed tRPC hook -> protected server procedure -> database/storage/provider -> typed response -> UI state. Keep that path intact. Do not add random REST endpoints or duplicate client-side fetch wrappers when a tRPC procedure already represents the feature.

1.4 4. How the server works

During development, the command `pnpm dev` starts the Express server through `server/_core/index.ts`. The server hosts the API and the Vite development experience. In production, `pnpm build` creates the frontend bundle and server bundle, and `pnpm start` runs the compiled server. Never hardcode a port; the hosting environment supplies it.

A request reaches `/api/trpc`. The framework builds a request context containing the current user when a valid Manus OAuth session cookie exists. A `publicProcedure` can be used without a signed-in user. A `protectedProcedure` requires a user. Admin procedures must check both authentication and the admin role or ownership record.

A safe procedure has five stages:

1. Validate the input with a schema such as Zod.
2. Authenticate the user and check authorization.
3. Read current data from the database rather than trusting browser values.
4. Perform the change in a transaction where multiple records must agree.
5. Return a typed result and log important failures without exposing secrets.

1.5 5. How the database should work

RideFlow uses a relational database. A relational database stores structured records in tables and links those records using keys. It is a good fit because users, vehicles, rides, payments, and documents have clear relationships.

The current schema includes auth-backed users, RideFlow profiles, driver onboarding information, uploaded-file metadata, fare rules, fare quotes, money-ledger entries, and admin settings. The next production schema should add or complete the following entities.

Table or domain	Important fields	Purpose
users	id, openId, email, role, timestamps	Identity and authorization root

Table or domain	Important fields	Purpose
profiles	userId, phone, photo, role-specific details	Customer and driver profile data
vehicles	driverId, make, model, plate, category	Vehicle used for a ride
driver_documents	driverId, type, storageKey, status, reviewerId	License, insurance, registration review
fare_rules	city, base, perKm, perMinute, minimum, commissionBps	Owner-controlled pricing
fare_quotes	riderId, distance, duration, amount, expiresAt	Time-limited upfront quote
rides	riderId, driverId, status, pickup, destination, times	Trip lifecycle
ride_stops	rideId, sequence, coordinates, address	Multi-stop rides
ledger_entries	rideId, type, amount, currency, reference	Financial audit trail
payments	rideId, provider, providerId, status	Stripe/M-Pesa payment state
payouts	driverId, amount, providerId, status	Driver earnings disbursement
ride_events	rideId, actorId, eventType, payload, createdAt	Audit and support history
safety_contacts	riderId, contact, consent	Trip-sharing recipients
messages	rideId, senderId, body, sentAt	In-app chat

Use UTC timestamps at the API/database layer. Convert them to local time only when displaying them. Add indexes for fields used in lookups, such as user IDs, ride status, driver availability, provider transaction IDs, and quote expiry. Never store card numbers or file bytes in normal database columns.

1.5.1 Migrations

Change `drizzle/schema.ts` first. Run `pnpm drizzle-kit generate`, review the generated SQL, and apply that SQL through the project database migration workflow. Never apply an unreviewed destructive migration. Creating tables is safer than dropping or renaming them. Take a database backup before structural changes in production.

Money should be stored as integer minor units, for example whole KSh when the provider settles in shillings, or cents for a currency that uses cents. Never use floating-point arithmetic for money. The server calculates the 5% commission as $\text{eligibleFare} * 500 / 10000$ when the commission is represented as 500 basis points. Round according to one documented rule and record the exact amounts used.

1.6 6. How the RideFlow fare model works

RideFlow currently uses database-backed Nairobi fare rules with a safe code fallback for development. A quote includes the distance, estimated duration, base fare, distance charge, time charge, minimum-fare protection, safety/service fee, rider total, driver portion, and five-percent platform commission.

A conceptual calculation is:

```
metered = baseFare + distanceKm * ratePerKm + durationMinutes * ratePerMinute
eligibleFare = max(metered, minimumFare)
platformCommission = round(eligibleFare * 0.05)
riderTotal = eligibleFare + safetyFee + optional tip
driverGross = eligibleFare - platformCommission
```

The exact business rule must be written in the fare-rules table and shown in the admin panel. “KSh 50 cheaper” should not be implemented as an unconditional subtraction from every competitor fare. It can produce losses on short trips, underpay drivers, or violate a minimum fare. A safer approach is to set an explicit target fare model, compare it to public market observations, enforce a minimum, show the fee breakdown, and review the model using real completed-trip data.

Quotes should expire. The server should refuse to accept an old quote without recalculating. When a ride is completed, the final ledger should record the quote, any approved adjustment, the commission, the driver share, tips, refunds, and provider references.

1.7 7. Riders, drivers, and dispatch

A real ride flow has distinct states:

```
requested -> searching -> offered -> accepted -> driver_arriving
-> driver_waiting -> in_progress -> completed
```

Cancellation, rejection, timeout, safety escalation, and payment failure are side paths. Every transition should be validated on the server. A driver must not accept two overlapping rides. A rider must not be able to mark a ride completed before the driver or system confirms the trip state.

Dispatch normally needs driver availability, location freshness, vehicle category, distance to pickup, driver preferences, and safety restrictions. Begin with a simple nearest-eligible-driver algorithm. Add fairness, driver choice, scheduled rides, and marketplace balancing only after the basic lifecycle is reliable.

For live location, use a realtime transport such as WebSockets or a managed realtime service. Do not write every GPS update into a permanent audit table. Store a current location with a freshness timestamp and periodically store a route summary. Restrict location access to the active rider, assigned driver, safety contacts when explicitly shared, and authorized support staff.

1.8 8. Authentication and admin ownership

Manus OAuth currently provides the sign-in session. RideFlow should not store a plaintext password. The initial admin is configured as njengastephen112@gmail.com, but admin control is transferable to another verified RideFlow user through the admin procedure.

Admin actions should require authentication, an admin role or ownership check, explicit confirmation for high-risk changes, and an audit record. High-risk actions include changing commission, changing payment accounts, transferring ownership, approving a driver, issuing a large refund, and disabling a safety feature. The admin control room should eventually include fare rules, notification recipient, driver verification queue, user suspension, ride search, refund workflow, payout review, incident management, and audit-log search. Never make an admin action depend only on a hidden frontend button. The server must enforce it.

1.9 9. File storage

Profile photos, licenses, insurance documents, vehicle registration, and lost-item photos belong in private object storage. The database stores metadata: owner, purpose, ride or driver ID, MIME type, size, storage key, review state, and timestamps. The browser should upload through a protected server procedure or a short-lived signed upload URL.

Validate size and MIME type on the server. A filename extension is not enough. Use private keys that do not reveal a user's identity. Serve files through short-lived signed URLs. Only the owner, assigned support staff, or authorized administrator should be able to retrieve sensitive driver documents. Add malware scanning before accepting arbitrary documents in production.

1.10 10. Payments and payouts

RideFlow does not yet have a completed production payment integration. Before processing money, choose a provider and verify business onboarding, currency support, marketplace rules, refunds, disputes, taxes, and payout requirements.

For Stripe, investigate Stripe Connect marketplace flows. The platform account collects or coordinates payment, connected driver accounts receive their share, and RideFlow records the five-percent platform commission. Use server-side `PaymentIntents` or the current provider-recommended marketplace flow. Verify webhooks, use idempotency keys, and never trust a browser "payment succeeded" message.

For Kenya, investigate M-Pesa Daraja business onboarding, STK Push, callback validation, transaction references, reversals, and supported payout methods. Keep consumer keys, secrets, passkeys, and callback credentials in server-side secrets. Design a provider adapter so RideFlow can support Stripe and M-Pesa without duplicating ride logic.

A payment adapter should expose concepts such as `createCharge`, `verifyCharge`, `refundCharge`, and `createDriverPayout`. Provider-specific response fields should be stored as references and raw event records, not mixed throughout the UI.

1.11 11. Notifications and email

RideFlow's owner notification workflow is designed to alert the configured admin when a new user is first persisted. Confirm whether the configured notification integration delivers email, push, or an in-product owner alert in the current environment. If actual email delivery is required, configure a verified email provider or the approved owner-notification connector and test delivery without putting credentials in GitHub.

A signup notification should contain the new user's name, email, role, and creation time, but not passwords, payment details, identity documents, or access tokens. Do not send sensitive driver documents as ordinary email attachments. Send a protected admin link instead.

Production notifications need retries, deduplication, templates, delivery status, and an unsubscribe or operational-notification policy where applicable.

1.12 12. Testing strategy

Run `pnpm check` for TypeScript and `pnpm test` for Vitest. Unit tests should cover pure fare calculations, minimum fare protection, commission rounding, and validation. Authorization tests should prove that ordinary users cannot change fares, transfer admins, read private documents, or create unauthorized ledger entries. Integration tests should cover successful quote creation, upload validation, signup notification dispatch, admin transfer, payment webhooks, refunds, and failed provider callbacks.

Add browser tests later for sign-in, driver onboarding, booking, quote expiry, safety sharing, and the admin control room. A screenshot confirms layout; it does not prove that the server protected a request. Use a separate test database or controlled fixtures. Never use real payment credentials or real identity documents in tests.

1.13 13. Security checklist

Keep `.env` files, provider secrets, OAuth secrets, JWT secrets, database URLs, private keys, and uploaded documents out of GitHub. Use the project secret manager and rotate a secret immediately if it is exposed. Turn the repository private, enable GitHub two-factor authentication, and grant the smallest repository permission needed.

Add rate limiting to login, quote creation, uploads, messages, and payment endpoints. Verify webhook signatures. Validate ownership on every file and ride lookup. Use HTTPS in every deployed environment. Log security-relevant events without logging tokens or full payment data. Set retention periods for location history, messages, identity documents, and support records.

Prepare incident procedures for account takeover, payment mismatch, driver safety incident, exposed document, lost phone, and database outage. The person on call needs clear authority to disable rides, suspend accounts, refund money, and contact emergency services.

1.14 14. Deployment and PC setup

On a PC, install Git, Node.js, pnpm, and a code editor such as VS Code. Clone the repository:

```
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install
pnpm check
pnpm test
pnpm dev
```

Do not copy production secrets into the repository. Add them through the hosting/project secret manager. The most important configuration areas are database connection, OAuth URLs, JWT/session signing, storage API, notification service, maps provider, Stripe or Daraja credentials, and the initial admin email. Use branches for changes. A simple workflow is `main` for reviewed code and `feature/<name>` for work. Before merging, run checks, review the diff, test the relevant flow, and save a checkpoint. Never use a destructive reset to recover a project; use a known checkpoint or revert a specific commit.

1.15 15. Backups and operations

Back up the database on a schedule and test restoring it. Object storage needs its own retention and recovery policy. A database backup without file-storage recovery is incomplete because document metadata would point to missing files.

Monitor server errors, database connectivity, queue delays, provider webhook failures, payment reconciliation, storage failures, and unusual signup or cancellation activity. Create a daily operational review: unresolved support cases, driver documents awaiting review, rides stuck in a state, payment mismatches, failed payouts, safety reports, and infrastructure alerts.

1.16 16. What is still missing before real launch

The following are required before RideFlow should accept real passengers or money: a tested maps and routing integration; real driver availability and dispatch; realtime chat and live location; production Stripe or M-Pesa integration; driver payouts; legal review and business registration; driver contracts and verification operations; privacy and data-retention policy; fraud and dispute handling; customer support; emergency and incident procedures; backups and monitoring; load and security testing; and a controlled pilot with a small group of verified drivers.

Do not advertise a “live” marketplace while these systems are still demos. The current project is a strong product foundation and a private working prototype, not a substitute for licensed transport operations or payment-provider approval.

1.17 17. A practical six-month learning and build roadmap

Stage 1: Web foundations. Learn HTML, CSS, JavaScript, TypeScript, Git, and browser debugging. Rebuild one RideFlow card and one form without copying code. Learn how state changes in response to a click.

Stage 2: React and APIs. Learn components, hooks, controlled forms, loading states, errors, HTTP, JSON, and tRPC. Add one small profile field end to end: UI, procedure, database column, validation, test.

Stage 3: SQL and security. Learn primary keys, foreign keys, indexes, joins, transactions, migrations, authorization, sessions, and least privilege. Practice writing a query that returns only the current user’s records.

Stage 4: Marketplace lifecycle. Implement ride creation, driver acceptance, cancellation, completion, quote expiry, and a test ledger using fake money. Do not integrate real payments until the state machine is stable.

Stage 5: Infrastructure. Learn object storage, queues, realtime connections, webhooks, monitoring, backups, and deployment. Simulate provider timeouts and duplicate webhook events.

Stage 6: Pilot operations. Complete legal and provider onboarding, verify a small driver group, run test rides, reconcile every payment, document incidents, and only then expand carefully.

1.18 18. The most important commands

```
pnpm install      # Install dependencies
pnpm dev          # Start development server
pnpm check        # TypeScript validation
```

```
pnpm test          # Run Vitest
pnpm build          # Build frontend and server
pnpm format         # Format source files
pnpm drizzle-kit generate # Generate reviewed migration SQL
```

Use `git status`, `git diff`, `git log`, and `git branch` before making changes. Commit small, understandable changes. Read the project README and this handbook before modifying infrastructure files.

1.19 19. Final operating principle

RideFlow is not one screen. It is a coordinated system: identity, permissions, profiles, documents, fare rules, trips, location, messaging, payments, payouts, support, and safety. Build each subsystem with a clear owner, a database model, a server contract, a UI state, tests, logs, and a recovery path. When you can explain what happens on success, failure, retry, cancellation, and abuse, you are thinking like the engineer who can safely operate the platform.

1.19.1 Immediate next actions

1. Clone `raiven427/ride-flow` on the PC and run `pnpm check` and `pnpm test`.
2. Open the Admin Control Room and confirm fare rules, commission, notification email, and ownership transfer behavior.
3. Choose one payment provider and one maps provider; complete sandbox integration before considering a real pilot.
4. Create a database-plus-storage backup plan and write the first incident-response checklist.
5. Complete a legal and compliance review for the launch country before accepting real users or funds.

1.20 References

[1]: <https://react.dev/learn> React documentation: Learn React [2]: <https://www.typescriptlang.org/docs/> TypeScript Handbook [3]: <https://trpc.io/docs> tRPC documentation [4]: <https://orm.drizzle.team/docs/overview> Drizzle ORM documentation [5]: <https://owasp.org/www-project-top-ten/> OWASP Top 10 [6]: <https://docs.stripe.com/connect> Stripe Connect documentation [7]: <https://developer.safaricom.co.ke/> M-Pesa Daraja developer portal [8]: <https://git-scm.com/book/en/v2> Git documentation and Pro Git book [9]: <https://vitest.dev/guide/> Vitest documentation [10]: <https://developer.mozilla.org/en-US/docs/Web/HTTP> MDN HTTP documentation